Kalibracija kamere

Tara Knežević,
mag. ing. mech.

Kako dobiti 3D podatke o sceni iz 2D slike?

Kako dobiti 3D podatke o sceni iz 2D slike?

Zanimaju nas 3D koordinate u odnosu na neki koordinatni sustav svijeta (npr. baza robota) izražene u mm

Kako dobiti 3D podatke o sceni iz 2D slike?

Potrebno je rekonstruirati podatke poznate iz slika u metrični zapis

- Kako bi to postigli zanimaju nas dvije glavne stvari:

1. Pozicija i orijentacija kamere u odnosu na koordinatni sustav svijeta (u našem slučaju baza robota) → **EKSTRINZIČNA KALIBRACIJA**

2. Način na koji kamera preslikava točke perspektivne projekcije iz svijeta na svoju slikovnu ravninu → **INTRINZIČNA KALIBRACIJA**

- **KALIBRACIJA KAMERE** = postupak kojim određujemo ekstrinzične i intrinzične parametre kamere

Intrinzična kalibracija

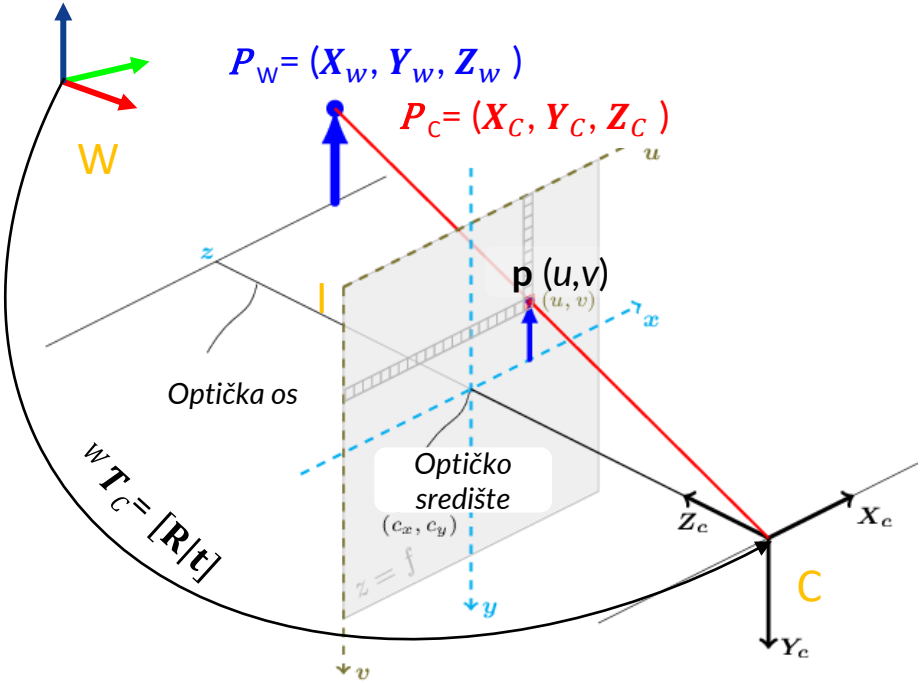
Intrinzični parametri:

1. **Žarišna duljina** - Predstavlja udaljenost leće od senzora
2. **Optički centar** - točka na senzoru kamere u kojoj se sijeku zrake svjetlosti koje prolaze kroz objektiv
3. **Koeficijenti distorzije** - opisuju nepravilnosti u optičkom sustavu kamere koje uzrokuju nesavršenosti u projekciji slike

Kako intrinzični parametri povezuju 2D prostor slike i 3D prostor svijeta?

- Intrinzični parametri kamere **projiciraju** 3D točku $(X_c[\mathrm{mm}], Y_c[\mathrm{mm}], Z_c[\mathrm{mm}])$ iz koordinatnog sustava kamere u 2D koordinatu slike $(u[\mathrm{px}], v[\mathrm{px}])$
- **Pinhole model kamere** opisuje perspektivnu projekciju bez leća i distorzije

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \underbrace{\underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{\text{MATRICA INTRINZIČNIH PARAMETARA}} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}}_{\text{INTRINZIČNI DIO PINHOLE MODELA KAMERE}}$$



Kako intrinzični parametri povezuju 2D prostor slike i 3D prostor svijeta?

- Z_c predstavlja udaljenost točke od kamere
- Dubina Z_c ne pojavljuje se eksplicitno u slici **(2D)**, ali utječe na **skaliranje projekcije** (udaljeniji objekti izgledaju manji)

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

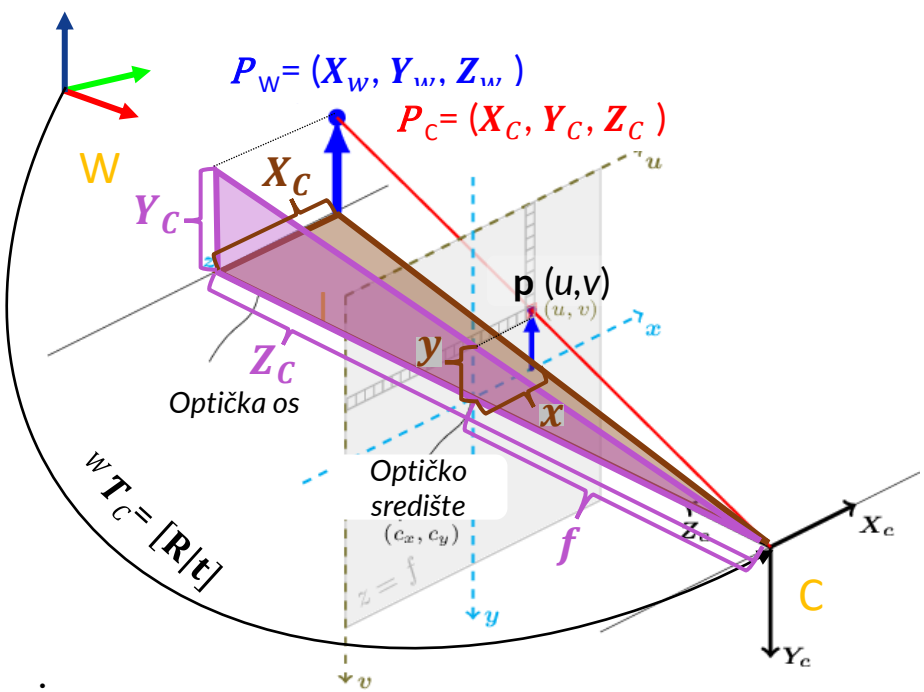
Sličnost trokuta uvodi ovisnost o Z_c , čime se dobiva **perspektivna projekcija**

$$\frac{X_c}{Z_c} = \frac{x}{f} \rightarrow x = \frac{1}{Z_c} f X_c$$

$$\frac{Y_c}{Z_c} = \frac{y}{f} \rightarrow y = \frac{1}{Z_c} f Y_c$$

$$u \neq x$$
$$v \neq y$$
$$f \neq f_x \neq f_y$$

Svi iznosi u prikazanim jednadžbama su u **[mm]!!!!**



u, v, f_x, f_y = ??

Kako intrinzični parametri povezuju 2D prostor slike i 3D prostor svijeta?

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

ZNAMO:

$$x[\text{mm}] = \frac{1}{Z_c[\text{mm}]} f[\text{mm}] X_c[\text{mm}]$$

$$y[\text{mm}] = \frac{1}{Z_c[\text{mm}]} f[\text{mm}] Y_c[\text{mm}]$$

TRAŽIMO:

$$u, v, f_x, f_y = ??$$

Kako bi se koordinate iz mm pretvorile u piksele, uvodimo veličine s_x i s_y ,koje određuju koliko milimetara odgovara jednom pikselu

$$u[\text{px}] = \frac{x[\text{mm}]}{s_x[\frac{mm}{px}]} \qquad v[\text{px}] = \frac{y[\text{mm}]}{s_y[\frac{mm}{px}]}$$

s_x= fizička veličina jednog piksela u horizontalnom smjeru [mm]
s_y = fizička veličina jednog piksela u vertikalnom smjeru [mm]

Parametri s_x i s_y mogu biti različiti jer pikseli na senzoru nisu nužno kvadratni

$$s_x \neq s_y$$

Moramo razlikovati žarišnu udaljenost po horizontalnom i vertikalnom smjeru:

$$f_x[\text{px}] = \frac{f[\text{mm}]}{s_x[\frac{mm}{px}]} \qquad f_y[\text{px}] = \frac{f[\text{mm}]}{s_y[\frac{mm}{px}]}$$

f_x = žarišna udaljenost u horizontalnom smjeru [px]
f_y = žarišna udaljenost u vertikalnom smjeru[px]

Kako intrinzični parametri povezuju 2D prostor slike i 3D prostor svijeta?

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

Kada uvedemo sve supstitucije, dobijemo izraze:

$$u[\text{px}] = \frac{1}{Z_c[\text{mm}]} f_x[\text{px}] X_c[\text{mm}]$$

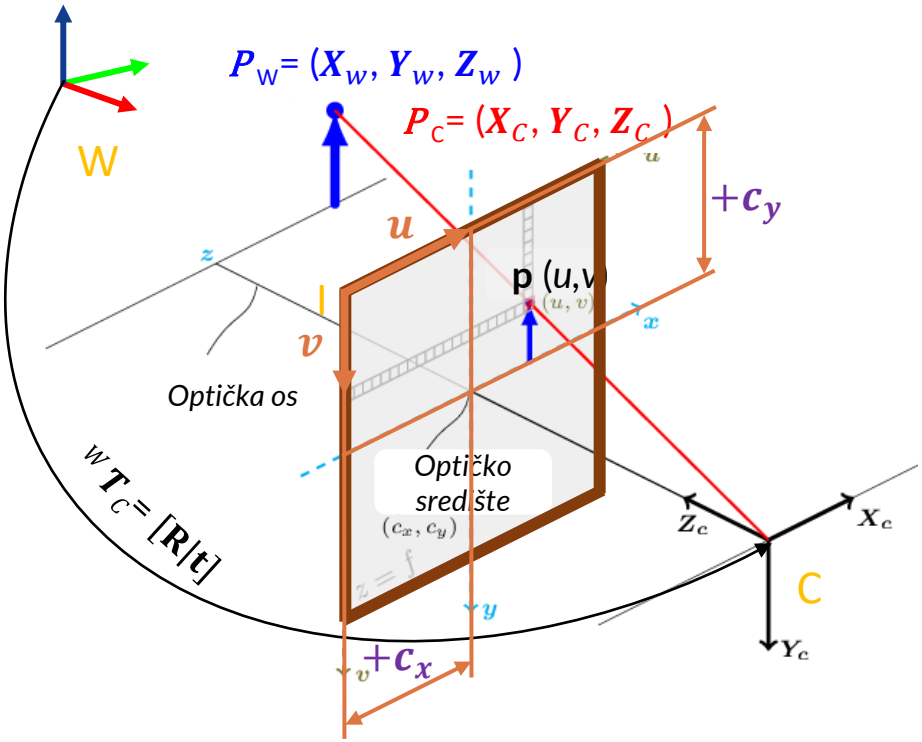
$$v[\text{px}] = \frac{1}{Z_c[\text{mm}]} f_y[\text{px}] Y_c[\text{mm}]$$

Što nedostaje?

Projekcija se događa u odnosu na **optičku os** (centar slike), dok je **koordinatni sustav slike** definiran u gornjem lijevom kutu, pa je potrebno uvesti pomak (c_x , c_y).

$$u[\text{px}] = \frac{1}{z_c[\text{mm}]} f_x[\text{px}] X_c[\text{mm}] + c_x[\text{px}]$$

$$v[\text{px}] = \frac{1}{z_c[\text{mm}]} f_y[\text{px}] Y_c[\text{mm}] + c_y[\text{px}]$$



Princip intrinzične kalibracije

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

Princip intrinzične kalibracije

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

očitamo s kamere – detektirane točke
[px]

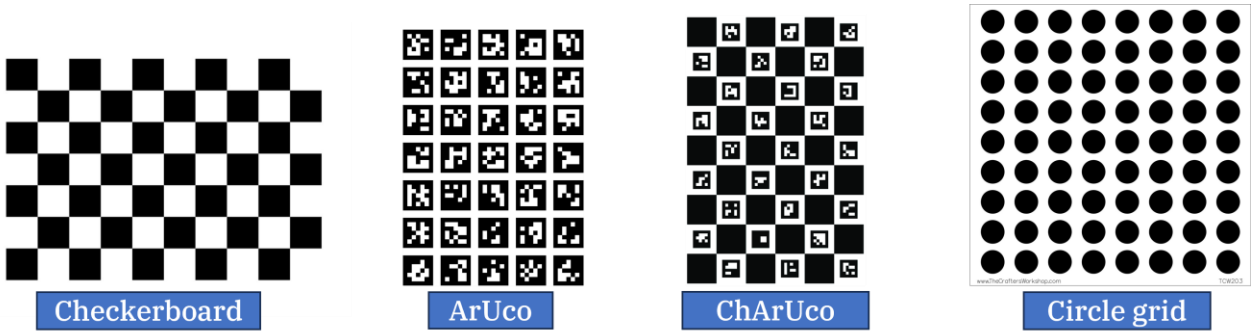
Princip intrinzične kalibracije

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

očitamo s kamere – detektirane točke [px]

poznata geometrija (ChAruco ploča)

- **Kalibracijske ploče** su posebno dizajnirani uzorci s poznatim rasporedom točaka, kvadrata ili markera
- One služe kao **referenca u prostoru**, jer su dimenzije i pozicije svih elemenata unaprijed poznate



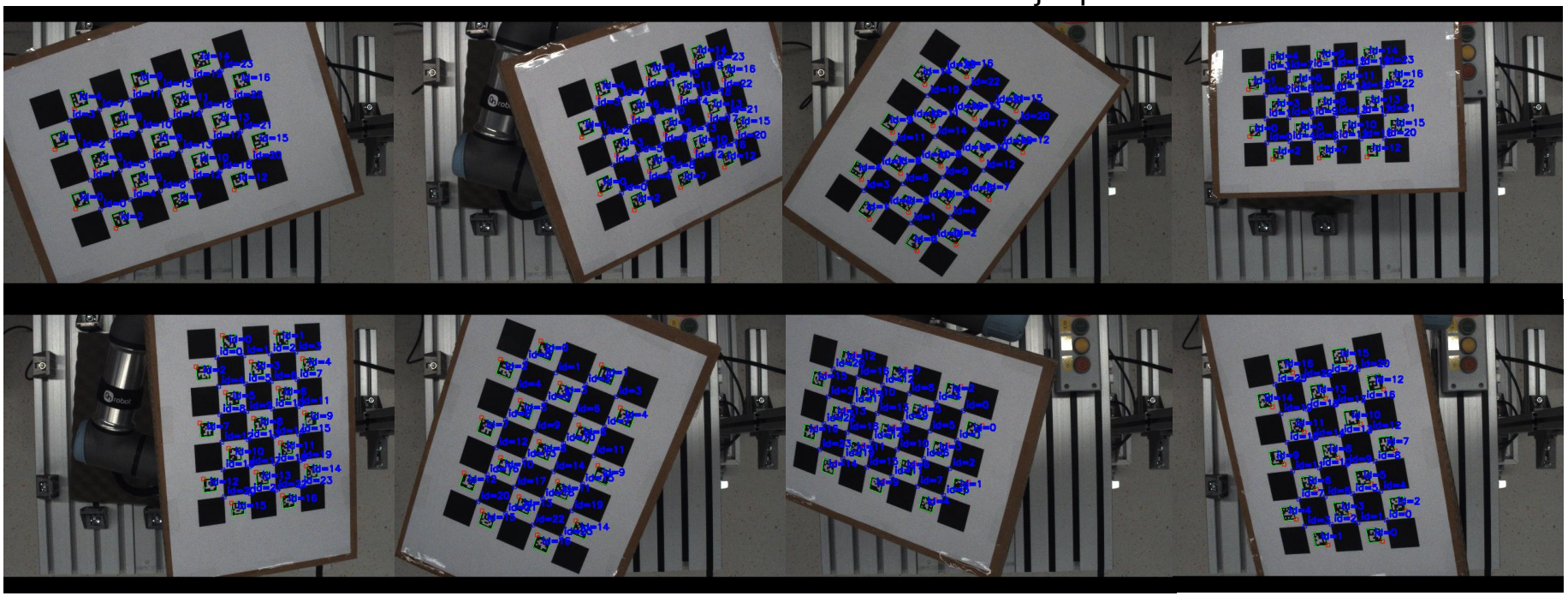
Princip intrinzične kalibracije

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

očitamo s kamere – detektirane točke [px]

poznata geometrija (ChAruco ploča)

Primjer pronađenih markera na ChAruco



Princip intrinzične kalibracije

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

očitamo s kamere – detektirane točke [px]

poznata geometrija (ChAruco ploča)

podaci koje tražimo – INTRINZIČNA MATRICA

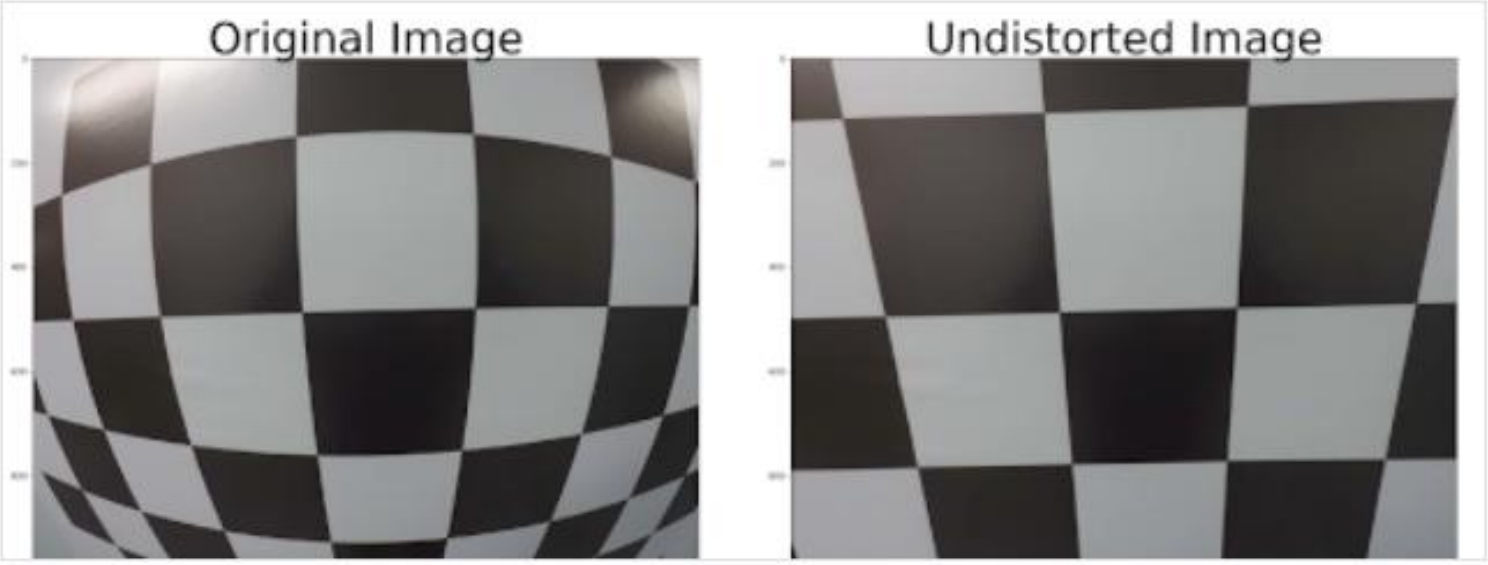
- PnP algoritam (Perspective-n-Point):
  - Poznate 3D točke projiciraju se na sliku i uspoređuju s detektiranim 2D točkama
  - Razlika između projekcije i mjerenja definira pogrešku (reprojekcijska pogreška)
  - Parametri kamere optimiziraju se tako da se ta pogreška minimizira

$$error = \sum_{i=1}^{n} \left\| \vec{p}_{det}{}^{(i)} - \vec{p}_{proj}{}^{(i)} \right\|^2$$

n = ukupan broj detektiranih točaka
$\vec{p}_{det}{}^{(i)}$ = detektirana 2D točka na slici
$\vec{p}_{proj}{}^{(i)}$ = točka dobivena projekcijom poznate 3D točke korištenjem trenutne procjene parametara

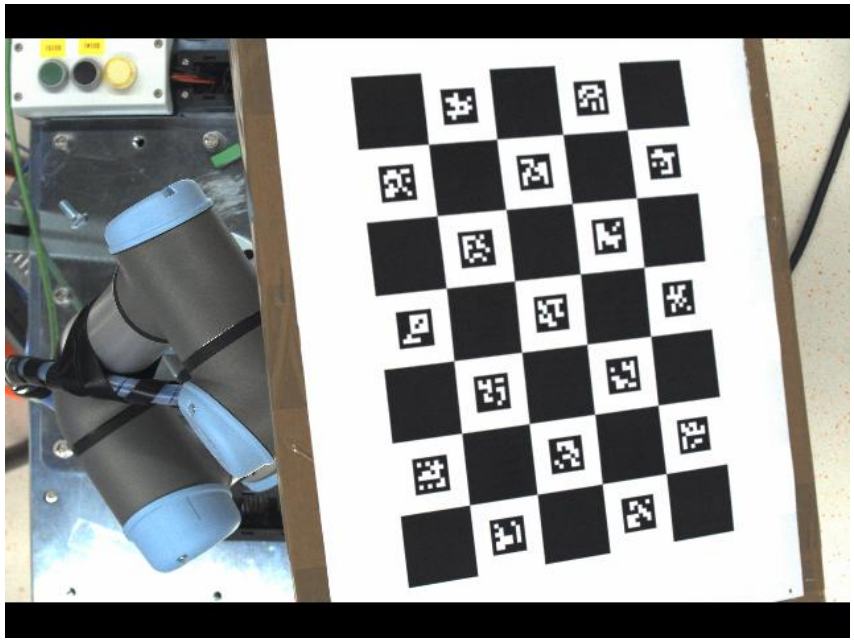
Zašto je to važno dobro poznavati intrinzične parametre?

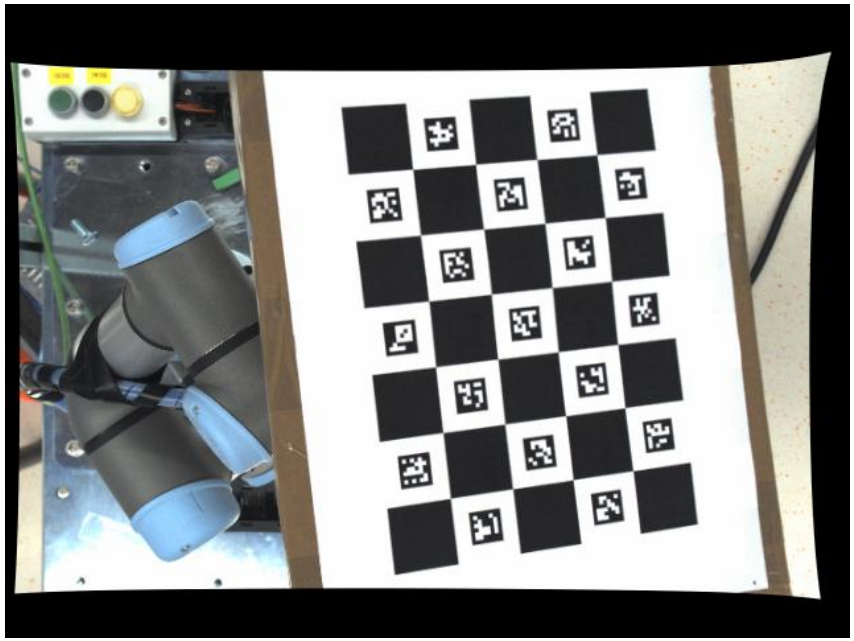
- Točna kalibracija i poznavanje intrinzičnih parametara omogućuje:
  - ispravljanje deformacija slike
  - pouzdanu detekciju i praćenje objekata
  - točnije određivanje udaljenosti i pozicije objekata u prostoru



Nakon intrinzične kalibracije:

- Nakon ispravka utjecaja distorzije na sliku, moguće je da će se dio informacija „izgubiti”





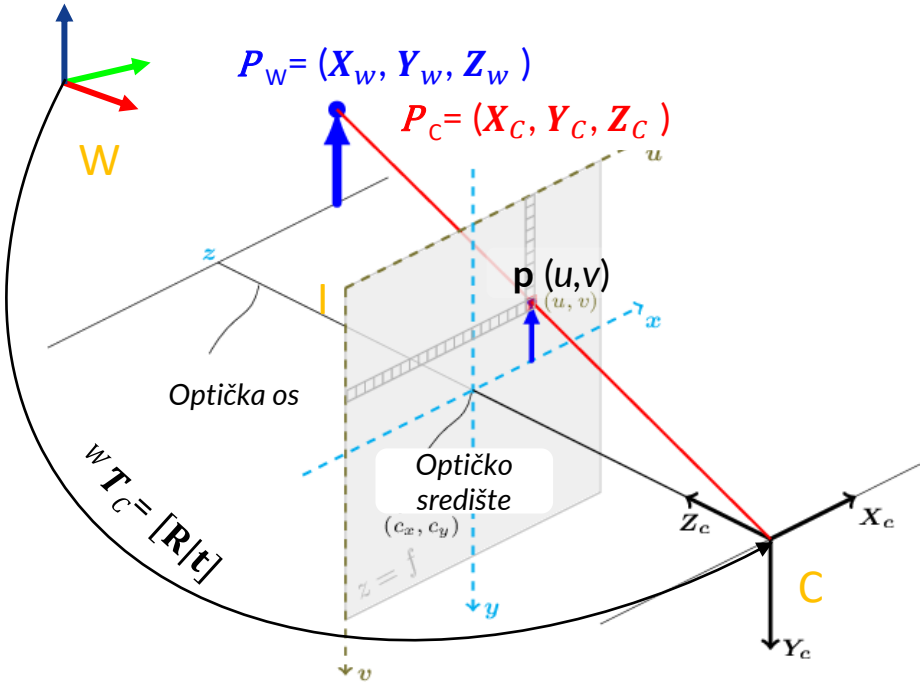
Ekstrinzična kalibracija

Pinhole model kamere – odakle dolaze X_c, Y_c, Z_c?

- Do sada smo se bavili pitanjem: kako pretvoriti 3D točku u kameri u 2D sliku?
- Stvarne točke su u svijetu (npr. baza robota)

Moramo odgovoriti na pitanje: Kako 3D točku iz koordinatnog sustava kamere izraziti u koordinatnom sustavu svijeta?

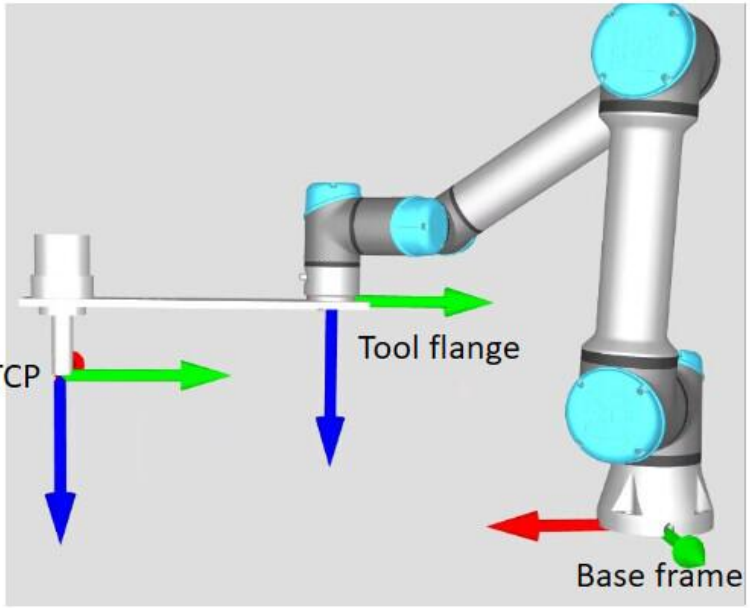
Kako dobiti (X_c, Y_c, Z_c) iz poznate točke (X_w, Y_w, Z_w)?



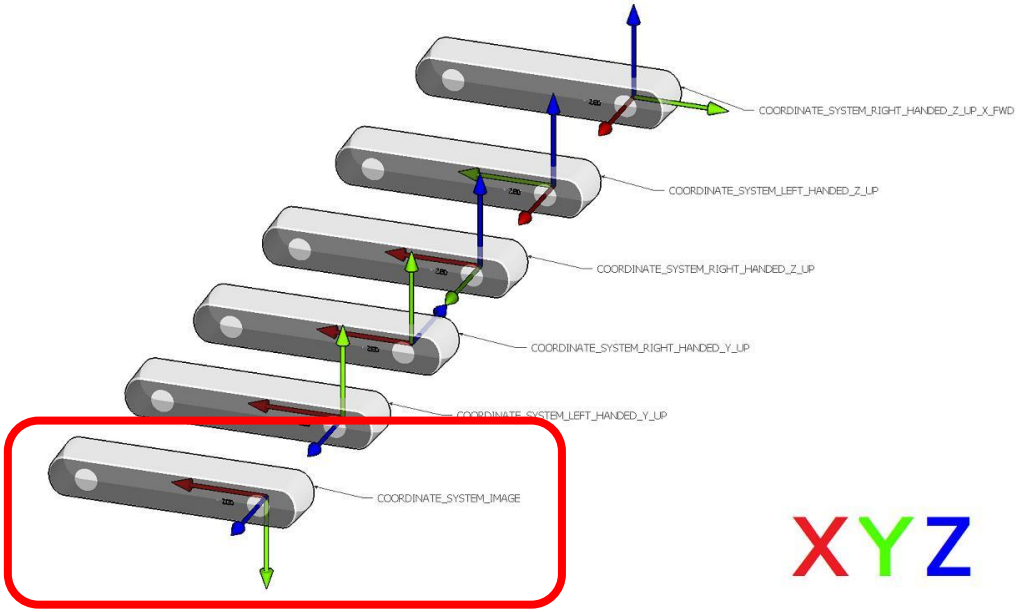
Napomena: nadalje ćemo koordinatni sustav svijeta (world) smatrati koordinatnim sustavom baze robota. Međutim, kada ne bismo imali robotsku ruku, ovaj postupak bi i dalje vrijedio za neki drugi koordinatni sustav svijeta

Koordinatni sustavi

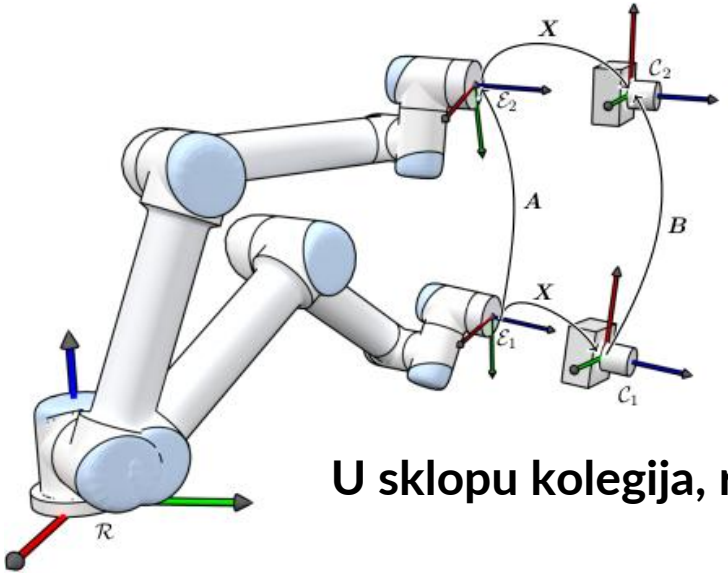
Robota

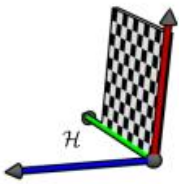


Senzora



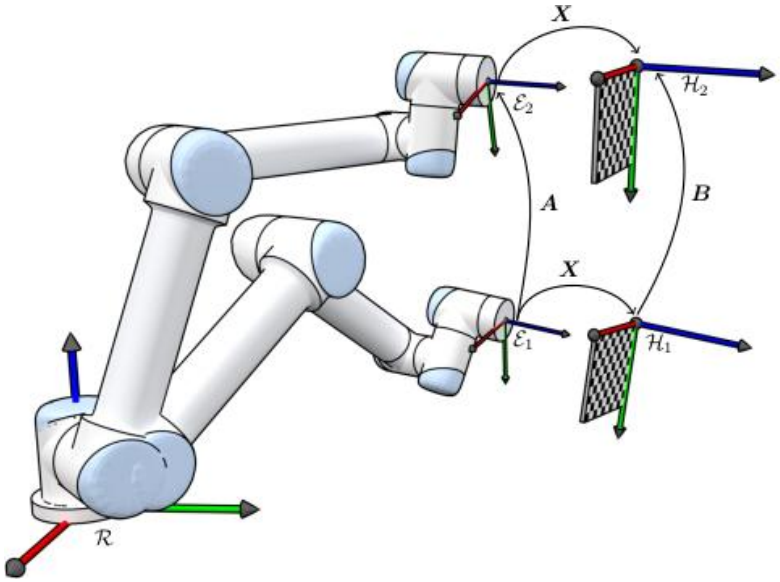
Moguće konfiguracije odnosa robot-kamera

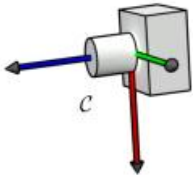




1. Hand in eye (kamera fiksno vezana na robota)

U sklopu kolegija, raditi će se HAND IN EYE kalibracija

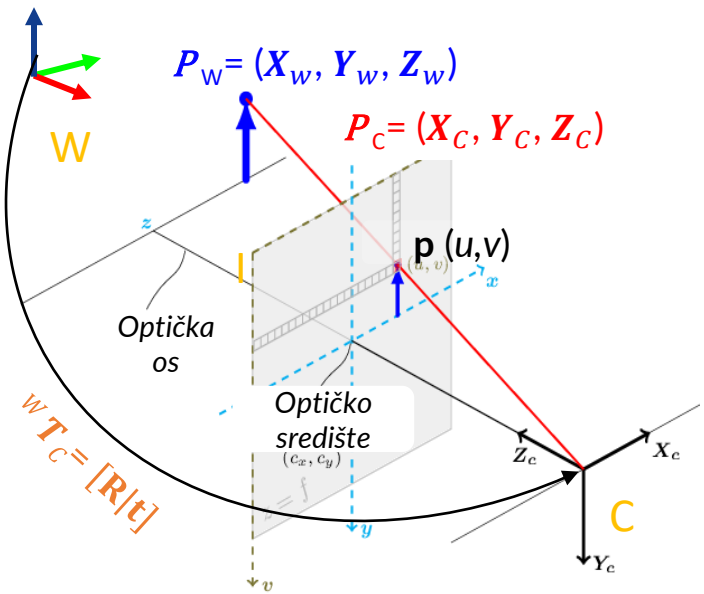




2. Hand to eye (kamera fiksno vezana u radnom prostoru robota)

Transformacija baza robota → kamera – kako dobiti X_c, Y_c, Z_c?

- Točke u prostoru su poznate u koordinatnom sustavu baze robota ($\boldsymbol{X_w}, \boldsymbol{Y_w}, \boldsymbol{Z_w}$)
- Potrebno ih je izraziti u koordinatnom sustavu kamere ($\boldsymbol{X_c}, \boldsymbol{Y_c}, \boldsymbol{Z_c}$)



$$\begin{bmatrix} X_C \\ Y_C \\ Z_C \\ 1 \end{bmatrix} = {}^W T_C \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

Ova transformacija opisuje gdje se kamera nalazi u odnosu na svijet

EKSTRINZIČNI DIO PINHOLE MODELA KAMERE

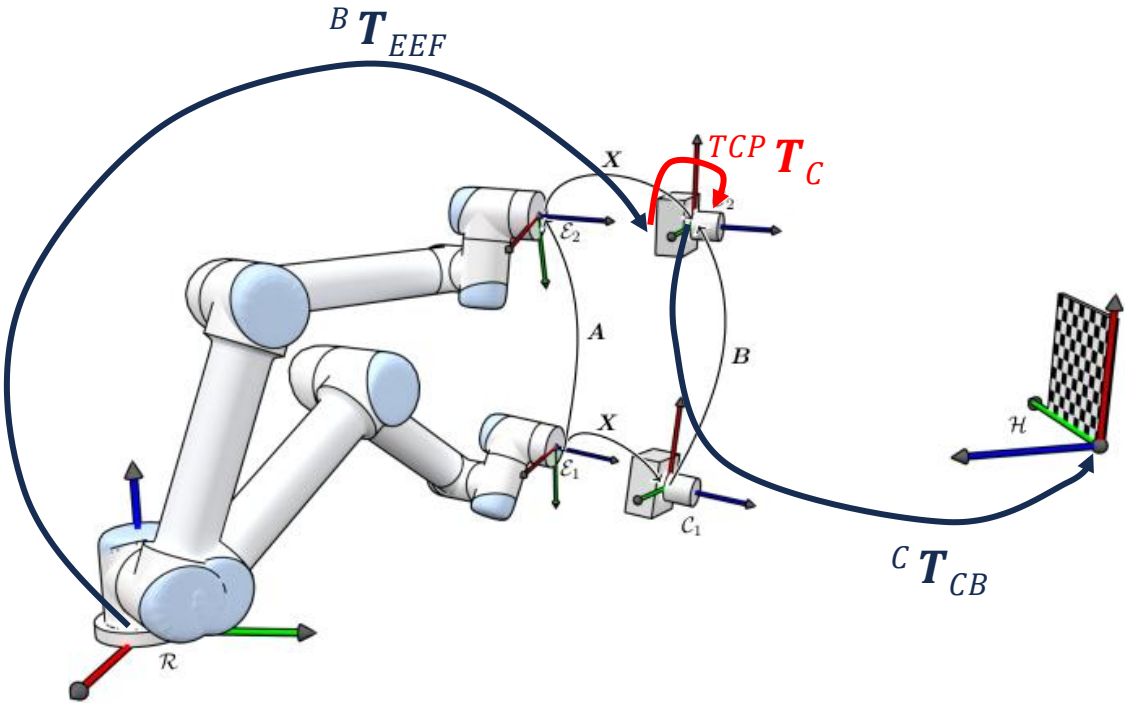
$$\underbrace{\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}}_{\text{INTRINZIČNI DIO PINHOLE MODELA KAMERE}} \longrightarrow \underbrace{\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_C} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \overbrace{[\mathbf{R}|\mathbf{t}]} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}}_{\text{PINHOLE MODEL KAMERE}}$$

Princip ekstrinzične kalibracije – Tsai metoda

- Tražimo fiksni odnos između kamere i hvataljke koji vrijedi za sve poze robota

- Ulazni podaci:
  1. Odnos baze i TCP-a robota
  2. Položaj ChArUco ploče u kameri

- Izlazni podaci:
  1. Odnos TCP-a robota i kamere



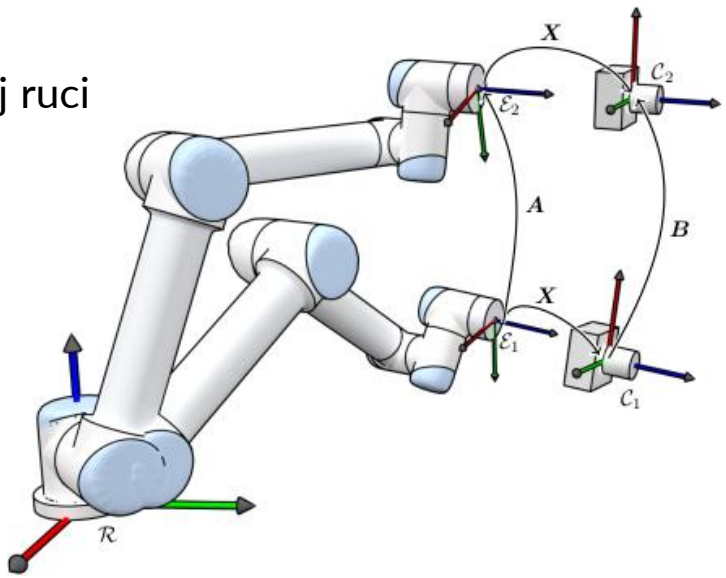
Princip ekstrinzične kalibracije – Tsai metoda

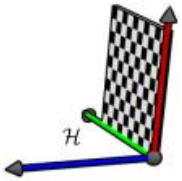
- Temeljna jednadžba hand-eye kalibracije: $AX = XB$

1. X = nepoznata transformacija ${}^{TCP}\boldsymbol{T}_C$
   - Konstantna vrijednost
   - Daje nam informaciju gdje je kamera montirana na robotskoj ruci
2. A = gibanje robota između dvije poze $\Delta^B \boldsymbol{T}_{TCP}$
   - kako se TCP pomaknuo iz poze i u pozu j

$$A = ({}^{B}T_{TCP}^{(i)})^{-1}\ {}^{B}T_{TCP}^{(j)}$$

Zašto inverz prve matrice?





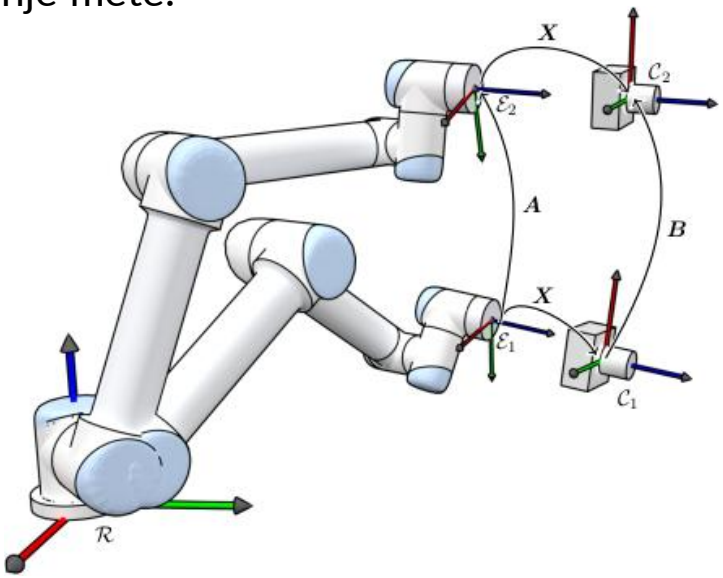
Princip ekstrinzične kalibracije – Tsai metoda

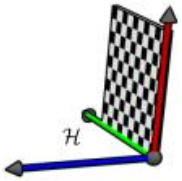
- Temeljna jednadžba hand-eye kalibracije: $AX = XB$

3. B = gibanje mete viđeno kamerom ${}^{C}\boldsymbol{T}_{CB}$

- Kamera u iste dvije poze vidi ChArUco ploču
- Iz toga se računa relativno gibanje mete:

$$B = ({}^{C}T_{CB}^{(i)})^{-1}\ {}^{C}T_{CB}^{(j)}$$



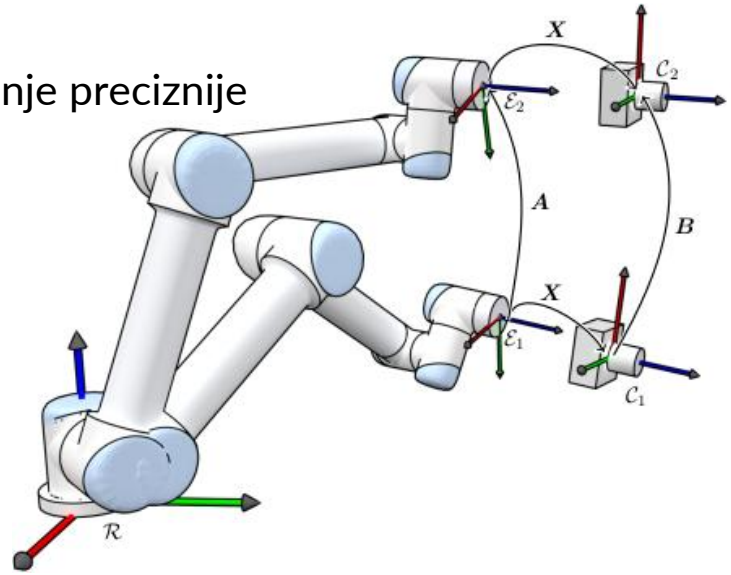


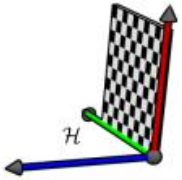
Princip ekstrinzične kalibracije – Tsai metoda

Tsai algoritam:

- Prvo računa rotacije, zatim translacije
- Uzima parove poza, a za svaki par dobijemo jednu jednadžbu $A_i X = X B_i$
- Što više jednadžbi imamo, to je rješenje preciznije

  Zašto?





- Zato što je hand-eye kalibracija **problem procjene nepoznatih parametara iz mjerenja**, a svaka dodatna jednadžba daje novu informaciju
- Robot ima malu grešku, kamera ima šum, detekcija ChArUco kutova nije savršena, kalibracija TCP-a nije savršena, kamera nije savršeno fiksna…

Zašto je to važno dobro poznavati ekstrinzične parametre?

Točna kalibracija i poznavanje ekstrinzičnih parametara omogućuje:

- Određivanje **položaja i orijentacije** kamere u odnosu na robota
- **Transformaciju koordinata** iz slike u svijet robota (npr. bazu)
- Izvođenje **manipulacije objektima** iz **okruženja** robota

Zadaci

Zadatak 1:

UVOD

- **Cilj**: Dobiti intrinzične parametre RGB kamere bez ručne kalibracije
- **Teorijska pozadina**:
  - Intel RealSense kamere već imaju ugrađenu kalibraciju
  - Moguće je direktno očitati:
    - rezoluciju slike
    - žarišne duljine (f_x, f_y)
    - optičko središte (c_x, c_y)
    - koeficijente distorzije
- **Alati**: koristi se **pyrealsense2**
- **Napomena**:
  - Nije potrebno provoditi kalibraciju (kamera je već kalibrirana)
  - Parametri se čitaju direktno iz uređaja

Zadatak 1:

UPUTA

- **Zadatak:**
  1. Pokrenuti RealSense pipeline za color stream
  2. Dohvatiti aktivni profil kamere
  3. Iz profila dohvatiti video stream profil
  4. Iz video stream profila očitati intrinzične parametre (pristupate varijabli **intr**)
  5. Konstruirati intrinzičnu matricu kamere (3x3) i vektor distorzije
     - np.array([*popunite*], dtype=np.float64)
  6. Ispisati:
     - Rezoluciju (u pikselima)
     - f_x, f_y (u pikselima)
     - c_x, c_y (u pikselima)
     - distorzijske koeficijente k_1, k_2, p_1, p_2, k_3
     - intrinzičnu matricu kamere
     - vektor distorzije
  7. Spremiti intrinzičnu matricu i distorzijske koeficijente kao .npy datoteke

Očekivano rješenje:

```
Intrinsics of Color Camera:
Resolution: 1280x720
Focal Length (fx, fy): 906.6868896484375, 905.9883422851562
Principal Point (ppx, ppy): 649.4818115234375, 371.50439453125
Distortion Coefficients: [0.0, 0.0, 0.0, 0.0, 0.0]

Intrinsic Matrix (mtx):
[[906.68688965   0.         649.48181152]
 [  0.         905.98834229 371.50439453]
 [  0.           0.           1.        ]]

Distortion Coefficients (dist):
[0. 0. 0. 0. 0.]
```

Utjecaj distorzije je kod RealSense kamera unaprijed ispravljen

Zadatak 2:

UVOD

- **Cilj**: Sinkronizirano prikupiti parove podataka potrebne za hand-eye kalibraciju (RGB kameru sa slike, pozu robota (TCP))
- **Teorijska pozadina**:
  - Za hand-eye kalibraciju potrebno je više različitih poza robota i pripadnih slika kalibracijske mete
  - Kamera slika ChArUco ploče
  - Robot (UR) šalje TCP pozu $[x, y, z, R_x, R_y, R_z]$ → potrebno pretvoriti u matricu transformacija

python -m pip install opencv-contrib-python

- **Alati**: koristi se **pyrealsense2, OpenCV, NumPy, socket**
- **Napomena**:
  - Svaka slika odgovara točno jednoj robot pozi
  - ChArUco ploča mora biti jasno vidljiva (CIJELA)
  - Poze robota moraju biti što raznolikije (različiti kutovi i udaljenosti)

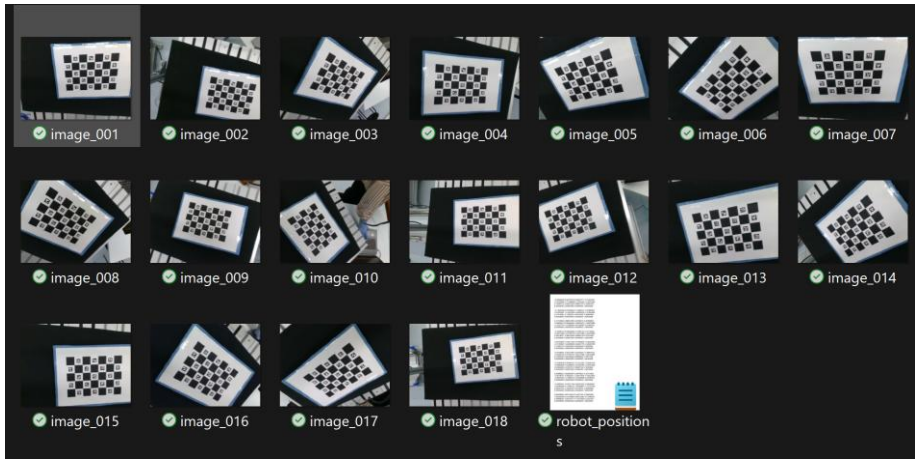
Kvaliteta kalibracije izravno ovisi o ovome

Zadatak 2:

UPUTA

- **Zadatak:**
  1. Pokrenuti RealSense pipeline za color stream
  2. Prikazati live sliku kamere (cv2.imshow)
  3. Namjestiti željenu pozu robota, Pritiskom na **SPACE** spremiti trenutnu RGB sliku
  4. Dohvatiti TCP pozu robota preko socket komunikacije
  5. Pretvoriti TCP podatke $[x, y, z, R_x, R_y, R_z]$ u homogenu matricu transformacija, 4x4 koristeći **cv.Rodrigues**
  6. Spremiti:
     - Sliku kao .png u data folder
     - robot pozu u robot_positions.txt
  7. Ponoviti postupak za više različitih poza robota
  8. Program završiti tipkom **Q** ili **ESC**

Očekivano rješenje:



Zadatak 2:

UPUTA točka 4 (dohvaćanje poze robota)

- **Zadatak:**
  - Pazite na IP adresu!

```
1   ▼ BeforeStart
2     var_2:=socket_open("ip_adresa",port)
3     Wait: 0.01
4     position_list:=[0,0,0,0,0,0]
5   ▼ Robot Program
6     Wait: 0.01
7     var_1:=get_actual_tcp_pose()
8     Wait: 0.01
9     position_list:=[var_1[0],var_1[1], var_1[2],var_1[3],var_1[4],var_1[5]]
10    Wait: 0.01
11    socket_send_line(position_list)
12    Wait: 0.01
13    socket_close()
14    Halt
```

Zadatak 3:

UVOD

- **Cilj**: Na temelju prethodno snimljenih slika odrediti položaj i orijentaciju ChArUco ploče u odnosu na kameru
  - Za svaku sliku izračunava se: translacija i rotacija mete te se prikazuju koordinatne osi mete na slici
- **Teorijska pozadina**:
  - ChArUco ploča kombinira ArUco markere (jedinstveni ID) i chessboard kutove (veća preciznost)
- **Alati**: koristi se **pyrealsense2, OpenCV, NumPy, glob, natsort**
- **Napomena**:
  - Za uspješnu procjenu poze potrebno je:
    - dovoljno vidljivih markera
    - minimalno 6 ChArUco kutova
    - dobra osvijetljenost slike
    - točni intrinzični parametri kamere

Zadatak 3:

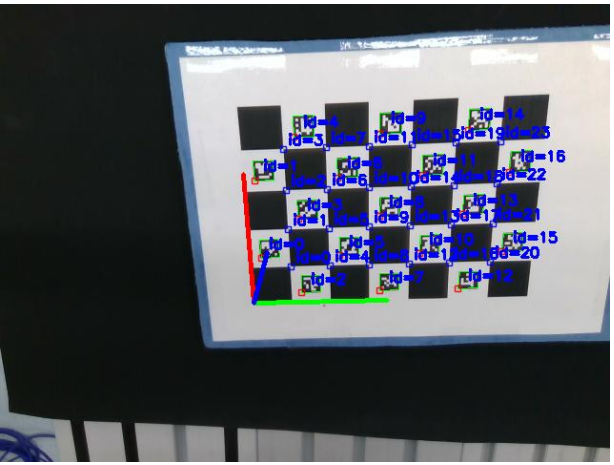
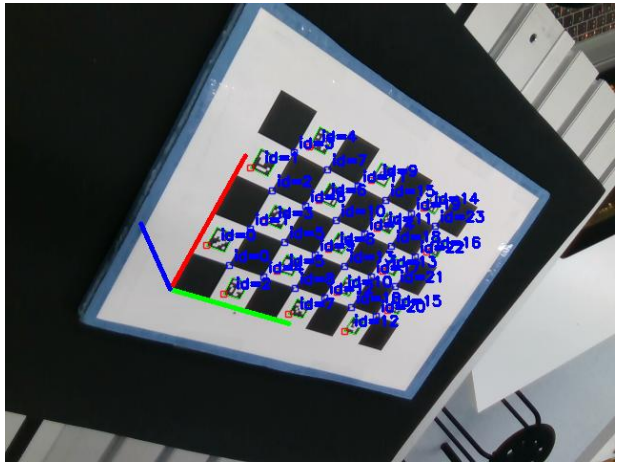
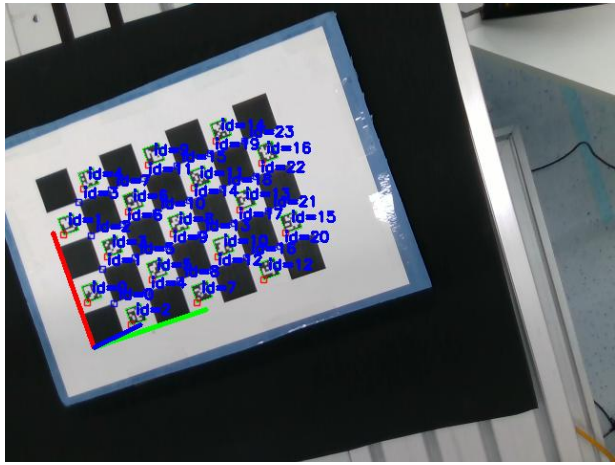
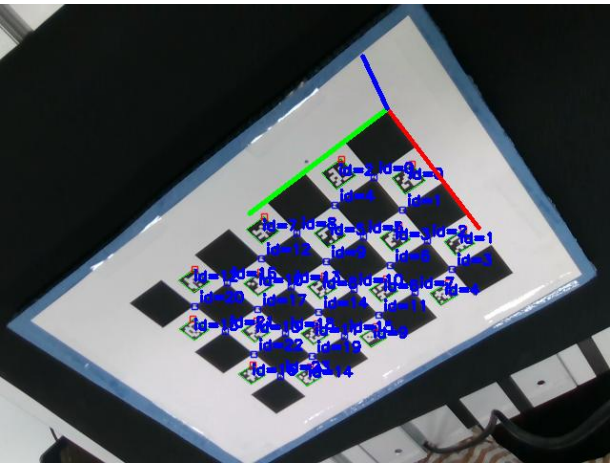
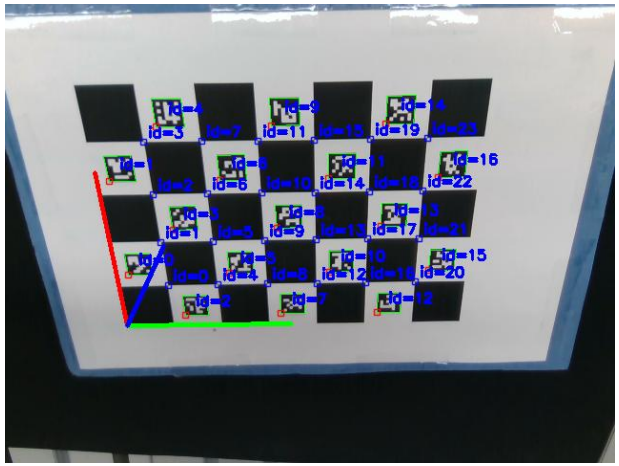
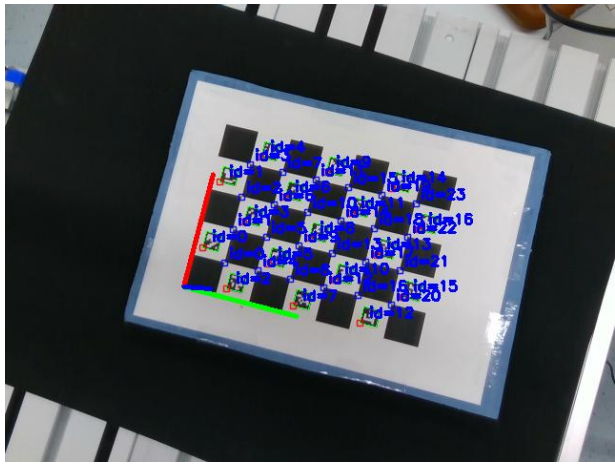
UPUTA

- **Zadatak:**
  1. Učitati intrinzične parametre kamere (npy. datoteke iz 1. zadatka)
  2. Definirati ChArUco ploču s poznatim dimenzijama (squareLength = 0.029 m, markerLength = 0.015 m)
  3. Učitati sve slike iz mape snimljene u 2. zadatku
  4. Za svaku sliku izvršiti detekciju ArUco markera (cv2.aruco.detectMarkers(...))
  5. Interpolirati ChArUco kutove (cv2.aruco.interpolateCornersCharuco(...))
  6. Ako je detektirano dovoljno kutova, procijeniti pozu ploče (cv2.aruco.estimatePoseCharucoBoard(...))
  7. Izdvojiti translaciju i rotaciju
  8. Nacrtati koordinatne osi mete na slici (cv2.drawFrameAxes(...))
  9. Spremiti rezultate
     - Tekstualne koordinate u koordinate.txt
     - Obrađene slike u dana/results mapu
  10. Prikazati rezultate i ponoviti za sve slike

Zadatak 3:

UPUTA

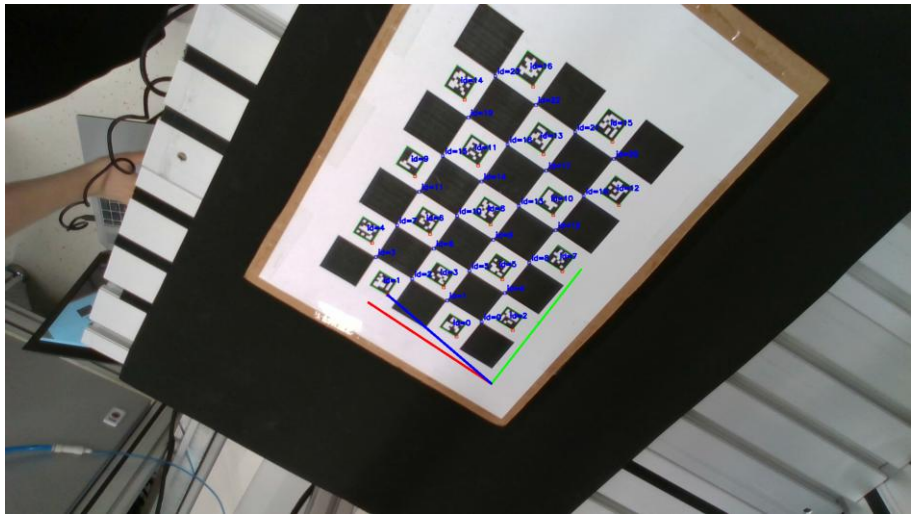
- Podaci koje želimo

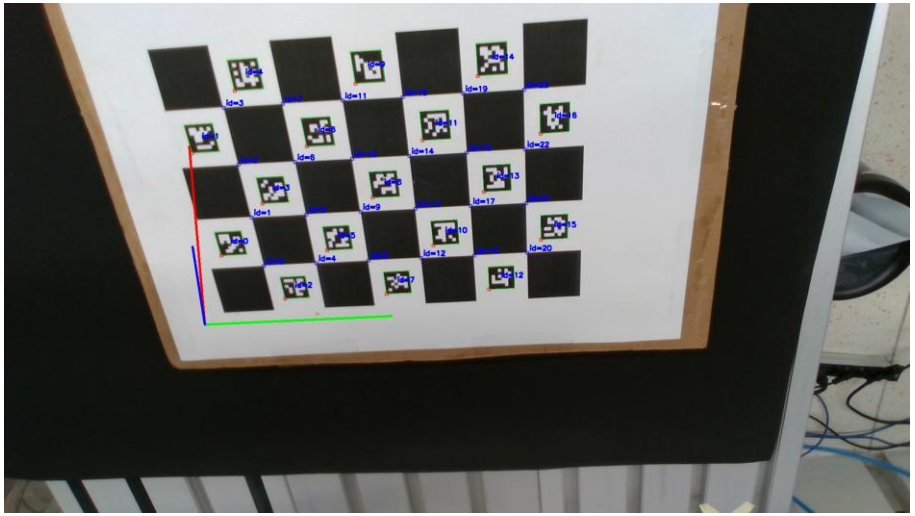







Zadatak 3:

UPUTA

- Podaci koje ne želimo





Zadatak 4:

UVOD

- **Cilj**: Odrediti točnu transformaciju između kamere i TCP-a robota korištenjem prethodno prikupljenih slika i robot poza
  - Rezultat kalibracije je odnos TCP-a i kamere
- **Teorijska pozadina**:
  - Hand-eye kalibracija rješava problem određivanja položaja kamere montirane na robotskoj ruci
  - Koristit će se Tsai-Lenz metoda: cv2.CALIB_HAND_EYE_TSAI
- **Alati**: koristi se **OpenCV, NumPy, glob, natsort**
- **Napomena**:
  - Ako dobijete krive rezultate potrebno se vratiti i provjeriti točnost zadataka 1, 2 i 3

Zadatak 4:

UPUTA

- **Zadatak:**
  1. Učitati intrinzične parametre kamere (npy. datoteke iz 1. zadatka)
  2. Učitati sve slike iz mape
  3. Učitati spremljene robot poze iz datoteke
  4. Definirati ChArUco ploču s poznatim dimenzijama
  5. Za svaku sliku detektirati ChArUco metu i izračunati pozu mete u odnosu na kameru
  6. Formirati parove podataka:
     - Poza robota (rotacija, translacija)
     - Poza mete (rotacija, translacija)
  7. Provjeriti broj valjanih parova (minimalno 5)
  8. Pokrenuti hand-eye kalibraciju (cv2.calibrateHandEye(...))
  9. Koristiti Tsai-Lenz metodu (cv2.CALIB_HAND_EYE_TSAI)
  10. Dobivenu transformaciju pretvoriti u 4x4 matricu
  11. Izračunati i inverz te spremiti rezultate kao .npy datoteke:
      - T_tcp_from_cam.npy
      - T_cam_from_tcp.npy

Očekivano rješenje:

```
Broj slika: 18
Broj robot matrica: 18

Valjanih parova za hand-eye: 18

T_tcp_from_cam  (CAMERA -> TCP):
[[-0.415829 -0.796502 -0.438942  0.116175]
 [-0.909389  0.358941  0.210171 -0.023845]
 [-0.009847  0.486565 -0.873589  0.051649]
 [ 0.        0.        0.        1.      ]]
Translacija [m]: [ 0.116175 -0.023845  0.051649]

T_cam_from_tcp  (TCP -> CAMERA):
[[-0.415829 -0.909389 -0.009847  0.027133]
 [-0.796502  0.358941  0.486565  0.075962]
 [-0.438942  0.210171 -0.873589  0.101125]
 [ 0.        0.        0.        1.      ]]
Translacija [m]: [0.027133 0.075962 0.101125]

Spremljeno:
 - T_tcp_from_cam.npy
 - T_cam_from_tcp.npy
```

Zadatak 5:

UVOD

- **Cilj**: Odrediti 3D koordinatu objekta u koordinatnom sustavu baze robota korištenjem RGB slike, depth kamere, hand-eye kalibracije, trenutačne TCP poze robota. Klikom na objekt u slici izračunava se njegova stvarna pozicija u prostoru.
- **Teorijska pozadina**:
  - Za odabrani piksel (u, v) i dubinu Z računa se 3D točka u koordinatnom sustavu kamere

$$X_c = \frac{(u - c_z)Z}{fx} \qquad Y_c = \frac{(u - c_y)Z}{fy} \qquad Z_c = Z$$

  - Transformacija u bazu robota – koristi se odnos dobiven u hand-eye kalibraciji, a konačno rješenje je: ${}^{B}\boldsymbol{T}_{C} = {}^{B}\boldsymbol{T}_{TCP}{}^{TCP}\boldsymbol{T}_{C}$
  - Da dobijemo položaj predmeta u bazi robota: $p_B = {}^{B}\boldsymbol{T}_{C}\, p_C$

- **Alati**: koristi se **pyrealsense2, OpenCV, NumPy, socket**

Zadatak 5:

UPUTA

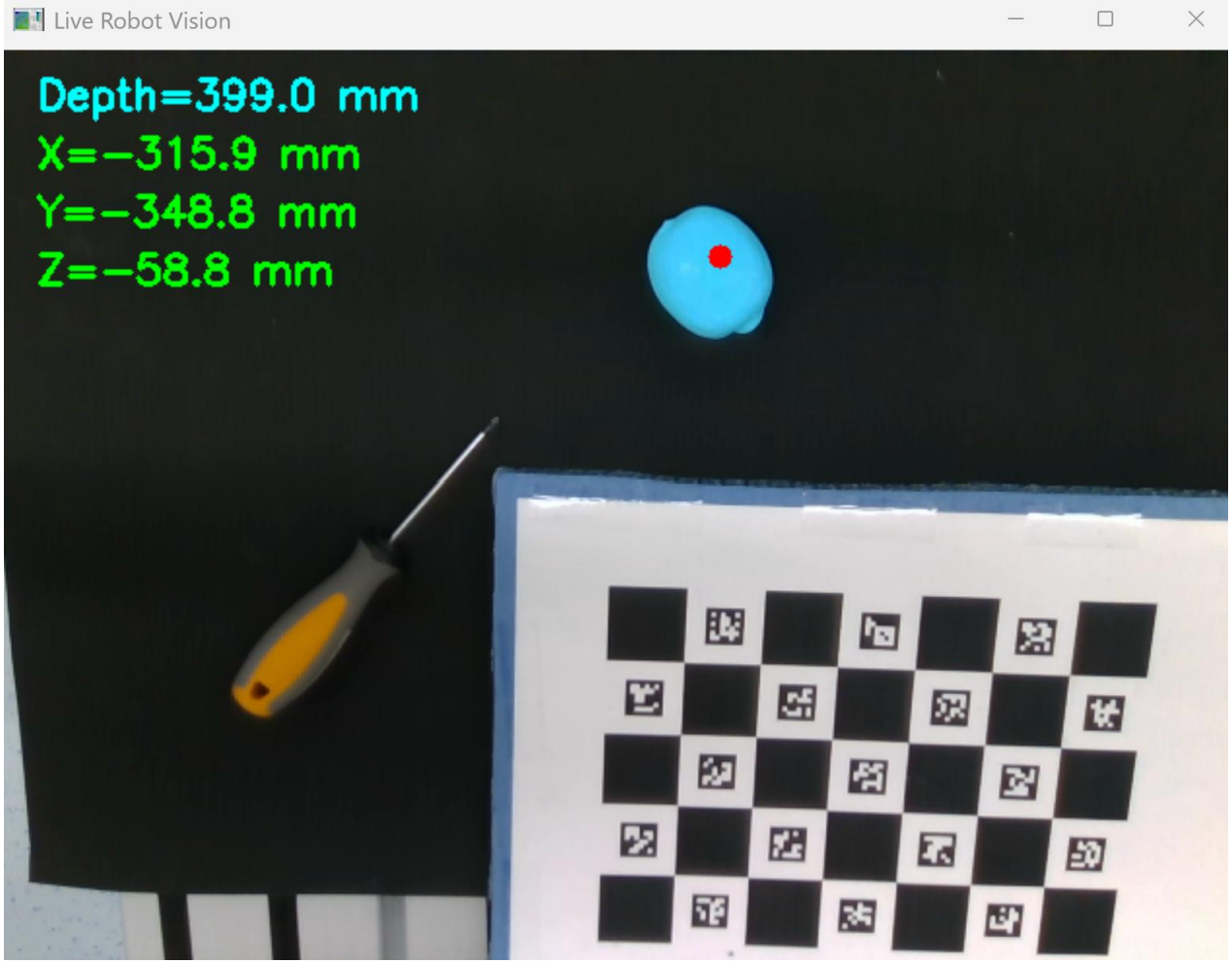
- **Zadatak:**
  1. Učitati intrinzične parametre kamere (npy. datoteke iz 1. zadatka)
  2. Učitati rezultate hand-eye kalibracije (npy. datoteke iz 4. zadatka)
  3. Pokrenuti RealSense kameru (color i depth stream)
  4. Poravnati depth sliku s RGB slikom (align = rs.align(rs.stream.color))
  5. Prikazati live sliku kamere
  6. Klikom miša odabrati objekt u slici
  7. Očitati depth vrijednost odabranog piksela (depth_frame.get_distance(u,v))
  8. Iz piksela (u, v)i depth podatka izračunati 3D koordinatu objekta u sustavu kamere
  9. Dohvatiti trenutačnu TCP pozu robota preko socket komunikacije
  10. Pretvoriti TCP pozu u homogenu 4x4 matricu (tcp_to_4x4(...))
  11. Primijeniti hand-eye transformaciju i izračunati koordinatu objekta u bazi robota
  12. Prikazati rezultate: dubinu objekta, X koordinatu, Y koordinatu, Z koordinatu
  13. Resetirati odabir tipkom **R**
  14. Završiti program tipkom **Q** ili **ESC**.
  15. Provjeriti valjanost rezultata s nastavnim osobljem

Očekivano rješenje:

```
[ROBOT] Čekam TCP...
TCP: [-0.219948, -0.260594, 0.33682, -0.355291, -0.306464, -1.97737]
Objekt u bazi [mm]: [-129.01527182 -472.96264233  -12.22164119]
```

Zadatak 5:

UPUTA



Zadatak 6:

- Prisjetimo se kolegija „Algoritmi strojnog vida”
- Potrebno je za jedan predmet skupiti 3D podatke (pointcloud) te sve potrebne podatke kako bi isti mogli registrirati u bazu robota